

If you want to read the other 4 weeks, check the other pdfs in this directory, I wrote in pencil there and decided to switch after not having my notebook for week 5 at the library

Emulator Week 5

Instructions, Decode, Addressing Nodes

The snes cpu, the 68512 is similar to the 6502 as noted before,

Instructions changes: Register sizes can be changed with the REP and SEP instructions. Taking an 8-bit immediate operand and resetting and setting the bits in the flag register. This is basically making it fast by set and reset multiple bits at the same time.

If the processor tries to run code that is meant for another type of size THIS WILL NOT WORK CORRECTLY. This will depend on the accumulator, so depend on that to assemble it correctly. Register size will determine this but the accumulator itself will always be 2 bytes.

There are a lot of little changes over the original 6502 that make life easier, and allow you to use fewer or smaller instructions. Most of these were actually introduced with the 65c02.

- **INA** - Increment accumulator.
- **DEA** - Decrement accumulator.
- **PHX** - Push X register.
- **PHY** - Push Y register.
- **PLX** - Pull X register.
- **PLY** - Pull Y register.
- **TXY** - Copy X register to Y register.
- **TYX** - Copy Y register to X register.
- **STZ** - Store zero. Can be indexed with X, and can be zeropage or absolute.
- **BRA** - Unconditional branch. This can save a byte over **JMP** but not a cycle, as both require exactly 3.
- **Indirect addressing** - Indexing is no longer mandatory on indirect addressing. For example you can write things like **LDA (\$00)**

Some cool instructions introduced here are JML, JSL and RTL that jump to a 24 bit address and do not the bank in the first 8 bits. These commands will be important later.

Fetch, decode loop:

From Gemini, could not find anything else in the library in the day I was there, about the Instruction Fetch Decode cycle: With fetch, the PC holds the next instruction's memory address and then the instruction is increased to do the next one after being translated into machine code. After that the decode part is that the IR (Instruction Register) to being placed there and then interpreted to correctly do the change it needs to do in that instruction, this is place in the Control Unit. This CU then signals the CPU to perform the change and can for example result into loading a value or changing the PC. This is repeated a lot during the amount the console is turned on and is extremely fast.

Good article about this from Near, not related to the specific instructions of SNES but helpful for ALU connected in the fetch decode cycle

<https://github.com/higan-emu/emulation-articles/tree/master/cpu/alu>

Addressing Modes: This is the way the cpu knows how to find the data it needs for example instructions. In the SNES some inputs got changed in order to accommodate the 24 bits needed to hold the instructions, in particular, `LDA`, `STA`, `ADC`, `SBC`, `ORA`, `AND`, `EOR`, and `CMP`, this also allows for the introduction of two forms of zeropage based in 24 bits.

With this, the stack pointer also got changed to be able to hold 64kb of the address space and hold it somewhere in the RAM. Remember that the stack points to the next available spot

Very important to note, the emulation mode is supposed to start up recreating the 65c02 and changes some of the processor basis, where some instructions take longer. Often can be ignored because it is used to run old programs meant for the 6502.

The decimal mode is a way to have it display things like money or specific scores that were not readily available on the NES.

Read later:

<https://archive.org/details/0893037893ProgrammingThe65816/page/107/mode/2up>

There is some pages talking about the addressing modes here, also it has the instructions, there are versions for complex and simple.

The code objective this week should be to implement the fetch instructions to the cpu and allow it to iterate by itself? I also think that some instructions from the SNES are good

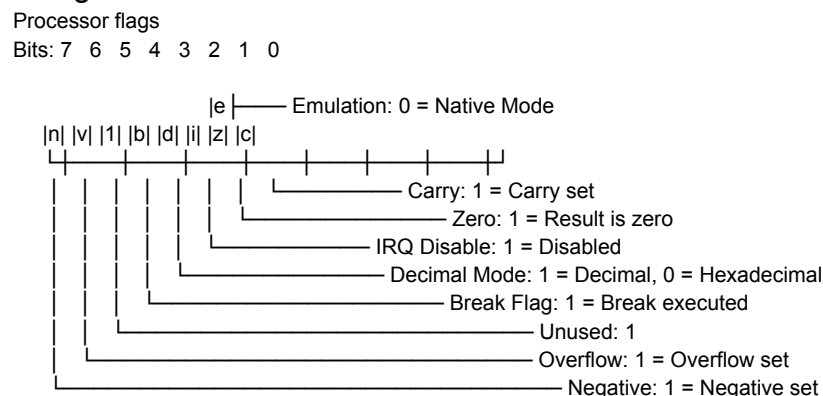
Emulator Week 6

Status Flags, Register Widths, and Emulation Mode

So the SNES cpu, the 65c816 has a cool thing called the emulation mode and the native mode. What the first one does is that it allows it to be compatible with NES systems and it is similar to the 6502, the NES CPU meanwhile Native Mode allows for the system to use its full capacity of power with 16 bits. This is done using the SEP and REP opcodes that constantly switch the processor flags in the future.

The variables set in the struct for flags are those that get changed, written in week 2 these are:

- Emulation (E): As described earlier, a way to allow or not NES games into the console. This has a different set of flags from the ones below. This is a diagram to those flags



- Negative (N): This one doesn't really affect much other than some branches in the SNES when those opcodes are called

- Overflow (V): Only 3 opcodes affect the calculation, ADC, SBC and BIT
- Accumulator (M) (Big one this week): This is what determines when 8 bits or 16 bits at the accumulator. This allows to use fewer instructions compared to that in the NES. This doesn't limit the 8 bits to only work as such, whenever 16 bits are enabled you can play with the 8 bits
- Index Register (X) (Y): This allows the X and Y registers to be set to 8 or 16 bits when either are both turned on for 8 bits or off for 16 bits
- Decimal mode (D): This will enter the decimal mode of the SNES meaning that for example add \$03 to \$09, the result is \$12 instead of \$0C.
- IRQ Disable (I): dont know how important it is but is a hardware thing
- Zero (Z) : Most opcodes modify this flag, but not of that much importance apparently
- Carry (C): Just how addition and subtraction are done

Continue reading later in the week (Thursday)